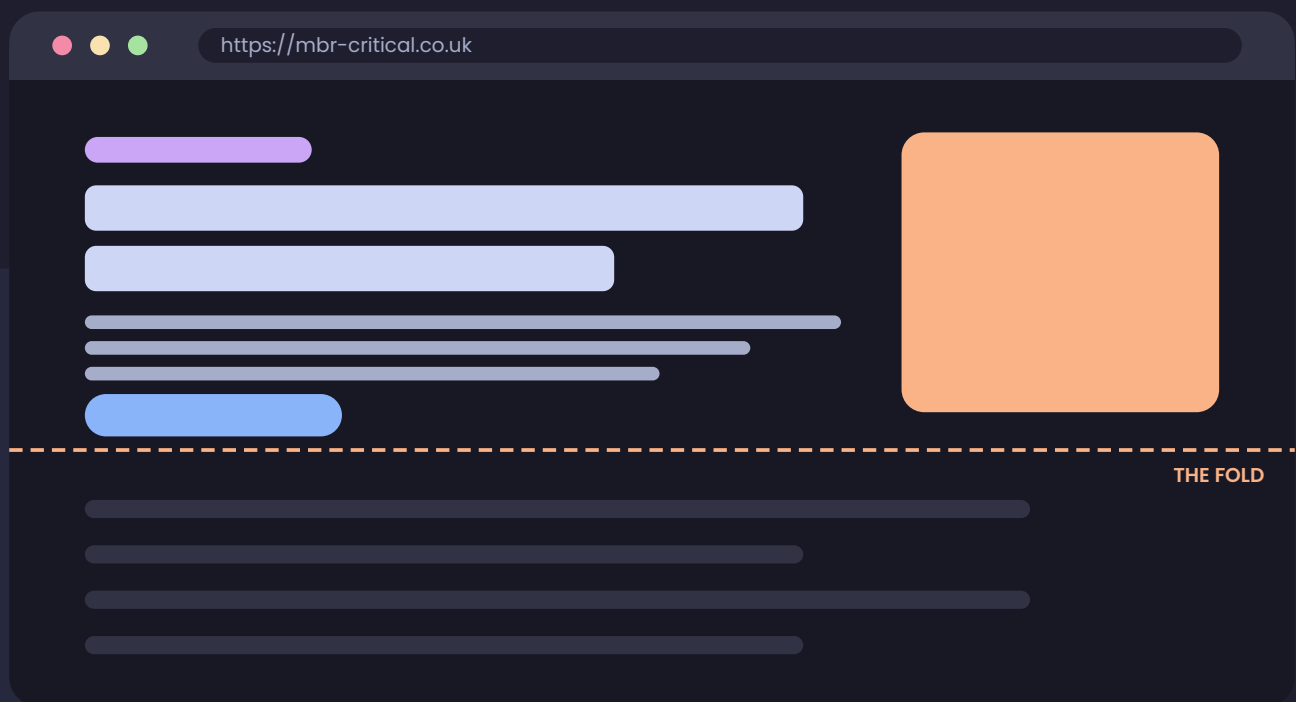


Critical CSS Generator

The User Guide

What critical CSS is, how the generator works, and how to turn its output into a faster first paint, with or without MBR Performance.



mbr-critical.co.uk

Guide for generator version 1.2.1 · October 2026

Free · No signup · Your browser does the heavy lifting

By Robert Palmer · littlewebshack.com

Contents

Everything you need to go from a slow first paint to an instant one.

1	Welcome	3
2	What is critical CSS?	4
3	How the generator works	6
4	Using the generator	7
5	Choosing viewports	8
6	Putting critical CSS to work	9
7	Using it with MBR Performance	10
8	Case study: four 100s	11
9	Tips, limits and good habits	12
10	Troubleshooting & FAQ	13
11	Glossary	14
12	Host your own generator	15
13	Don't have MBR Performance yet?	16
14	More from Little Web Shack	17

1 START HERE Welcome

The Critical CSS Generator at **mbr-critical.co.uk** works out which CSS rules a page needs to draw everything visible on screen the moment it loads, and hands you exactly those rules, ready to inline in your page head. It is free, needs no account, and runs on ordinary shared hosting, because the hard part happens in *your* web browser.

This guide explains what critical CSS is and why it speeds pages up, how the generator arrives at its answer, how to use each control, and how to put the result to work, including in the free MBR Performance plugin for WordPress.

At a glance

Details	
What you give it	A public page URL plus a viewport width and height (or one of four presets).
What you get back	A block of critical CSS to copy or download, a summary of what was kept, notes about anything unusual, and a preview of what the generator saw.
Where the work happens	Our server fetches the page and its stylesheets; your browser renders the page and extracts the CSS. No headless browser, no third-party service.
Cost	Free. No signup, no upsell, no tracking.
Built by	Robert Palmer of Little Web Shack, the developer behind MBR Performance.

TIP

In a hurry? Jump to section 4 for the step-by-step walkthrough, then section 7 if you use MBR Performance.

2

THE IDEA

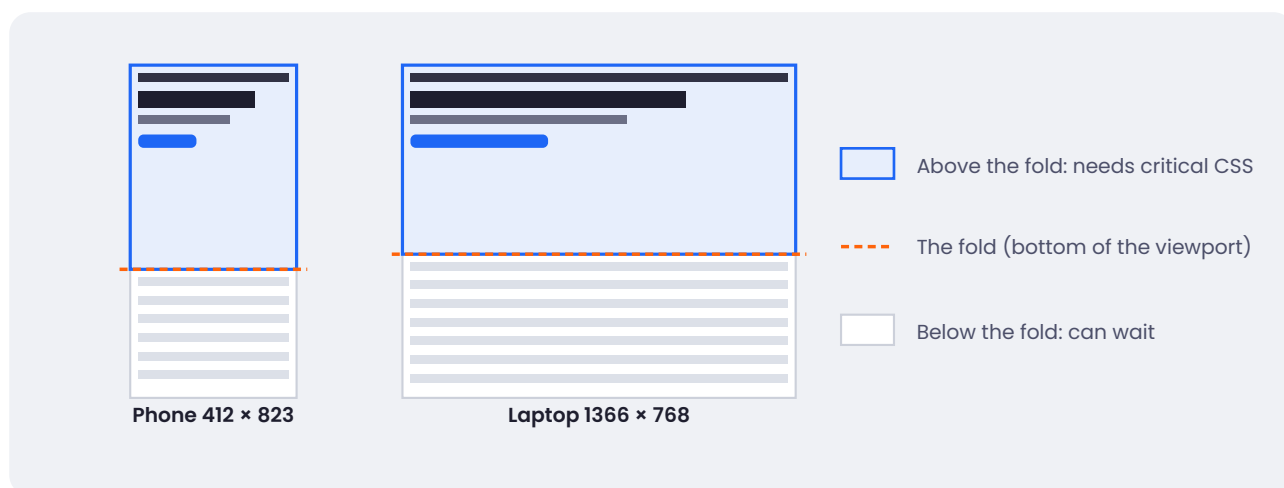
What is critical CSS?

Before a browser can paint anything, it has to download and process every stylesheet linked in the page head. Stylesheets are **render-blocking**: while they are in flight the visitor stares at a blank screen, even when the HTML has already arrived. On a fast connection that pause is barely noticeable. On a mid-range phone with patchy mobile data, a few large stylesheets can easily cost a second or more.

Yet most of those rules style things the visitor cannot see yet: the footer, the comments, the pricing table halfway down the page, the hover state of a button three screens away. **Critical CSS** is the small subset of rules needed to draw what is visible in the first screenful, the area web developers call **above the fold**.

The fold

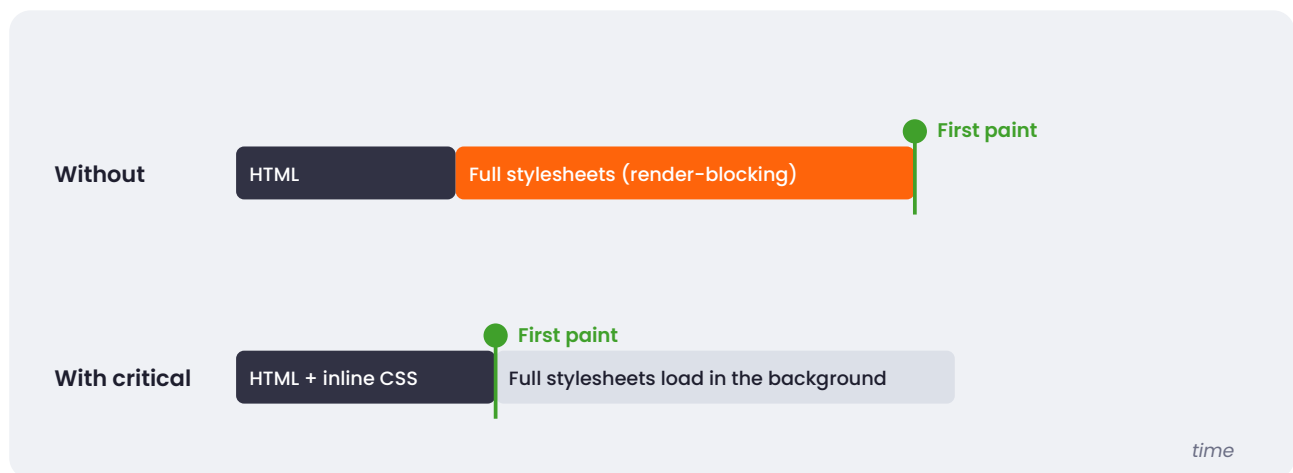
The fold is simply the bottom edge of the viewport when the page first loads. Its position depends on the device: a phone shows a narrow, tall slice of the page, a laptop a wide, shallow one. That is why the generator asks for a viewport size, and why critical CSS for a phone and for a desktop can be quite different.



The technique

The approach has two halves, and both matter:

- **Inline the critical CSS** in a `<style>` block in the page head. It arrives with the HTML, so the browser can paint the first screen immediately.
- **Load the full stylesheets without blocking**, so they download in the background and style the rest of the page by the time the visitor scrolls to it.



Illustrative only. The gap you gain depends on how much CSS a page loads and the visitor's connection.

Why it shows up in your scores

Google's Lighthouse and PageSpeed Insights flag stylesheets under *render-blocking requests*, and the delay feeds directly into two metrics:

- **First Contentful Paint (FCP):** when anything at all first appears.
- **Largest Contentful Paint (LCP):** when the biggest visible element, often a hero image or headline, finishes drawing. LCP is one of the three Core Web Vitals, and Google treats under 2.5 seconds as good.

NOTE

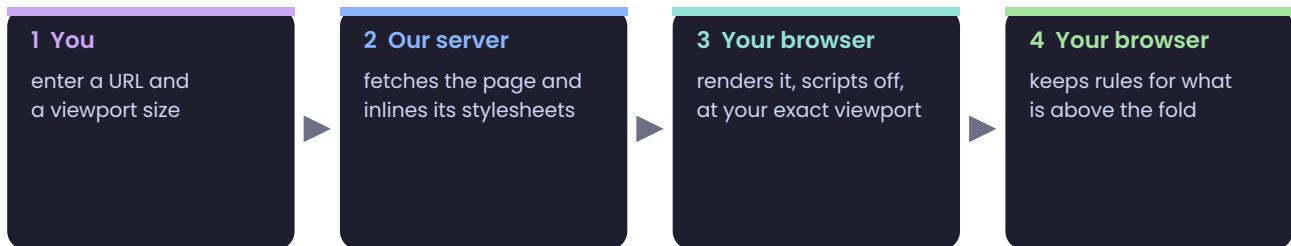
A common rule of thumb is to keep the HTML plus inlined critical CSS within roughly 14 KB after compression, because that is about what a server can send in the very first round trip of a new connection. It is a target, not a hard limit: a little over is still far better than a render-blocking stylesheet.

3

UNDER THE BONNET

How the generator works

Most online critical CSS tools run a headless copy of Chrome on a server for every request. That needs a dedicated machine, which is why such tools are usually paid or heavily rate-limited. This generator splits the job differently: the server does the fetching, and the browser you are already using does the rendering.



Step by step

1

Fetching

The server downloads the page HTML and every stylesheet it links to, `@import` files included, even those loaded with the common `async-CSS` tricks. Print-only stylesheets are skipped.

2

Preparing

Every `url()` is rewritten to a full address so fonts and background images still resolve. Scripts, event handlers and embedded frames are stripped out, and common lazy-loading attributes such as `data-src` are promoted so images still take up their real space. Any image or font address that points at a private network, or back at the generator's own site, is removed.

3

Rendering

Your browser loads the prepared page into a hidden, sandboxed frame sized to exactly the width and height you chose. Media queries, fonts and layout all behave as they would on a real screen of that size.

4

Extracting

The generator walks every CSS rule and keeps those whose selectors match an element visible above the fold. It keeps the matching `@media`, `@supports`, `@layer` and `@container` wrappers, plus `@font-face` rules for fonts in use and `@keyframes` for animations in use, each left exactly where it was defined.

5

Checking

The page is rendered again with *only* the critical CSS, and each element in the first view is compared with the full render for layout, colours, borders and fonts.

Smart details worth knowing

- **Hover, focus and decorations are kept** for elements above the fold: `a:hover` or `.btn:before` is kept when the element is visible, so nothing flickers.
- **Hidden things stay hidden.** For a closed mobile menu or an off-screen drawer, it keeps just the rules that hide it or hold it in place, so it never flashes into view.

4

WALKTHROUGH

Using the generator

Open mbr-critical.co.uk in any modern browser. The whole process takes a few seconds for a typical page.

1

Enter the page URL

Paste the full address of the page you want to optimise, for example <https://example.com/>. If you leave off the <https://>, it is added for you. The page must be publicly reachable: staging sites behind a login or a private network cannot be fetched.

2

Choose a viewport

Type a width and height in pixels, or tap a preset. Widths from 240 to 3840 px and heights from 240 to 2400 px are accepted. Section 5 explains how to pick.

3

Set the output options

Minify output strips whitespace for the smallest possible result; untick it for readable, indented CSS. **Include @font-face rules** keeps the font declarations for any font family used above the fold.

4

Generate

Press **Generate Critical CSS**. A status line tracks progress through fetching, rendering and extracting. Changing the output options afterwards re-extracts instantly, without fetching the page again.

5

Copy or download

Use **Copy** to put the CSS on your clipboard, or **Download .css** to save a file named after the site and viewport, such as [critical-example.com-1920x1080.css](#).

Reading the results

Part	What it tells you
Critical CSS	The rules to inline. Scroll the box to inspect them before copying.
Stats line	How many style rules were kept out of the total, the size of the critical CSS, and the total CSS found on the page, including inline <code><style></code> blocks. For example: <i>Kept 279 of 479 style rules: 44.5 KB of critical CSS, from 63.3 KB of CSS on the page.</i> On a lean landing page most rules may genuinely be critical.
Layout & style check	Compares position, size, visibility, colours, backgrounds, borders and fonts with the full render (not image pixels, shadows or transforms), for example: <i>212 of 215 above-the-fold elements match.</i>
Notes from this run	Appears only when there is something worth knowing: a stylesheet that could not be fetched, a limit reached, references that were removed, or elements that render differently with only the critical CSS.
Preview	A scaled-down picture of exactly what the generator rendered, inside a browser-window frame. If it looks wrong here, the CSS will reflect that, so it is a quick sanity check.

5

PHONES, LAPTOPS AND EVERYTHING BETWEEN

Choosing viewports

Critical CSS is only correct for the screen size it was generated at. A rule inside `@media (max-width: 768px)` simply does not apply on a 1920-pixel monitor, so a desktop run leaves it out, and phones then wait for the full stylesheet to get it.

The presets

Preset	Size	Good for
Mobile	375 × 812	A typical modern phone held upright.
Tablet	768 × 1024	Portrait tablets, and the layout break many themes use.
Laptop	1366 × 768	The most common laptop resolution; a short, wide fold.
Desktop	1920 × 1080	Full HD monitors, and the desktop layout of most sites.

TIP

Testing with PageSpeed Insights? Its mobile run emulates a phone of **412 × 823**. Type that in to match it exactly.

Covering every device

Because the generator keeps media query wrappers intact, you can safely **combine** runs. Generate once for mobile and once for desktop, then join the two outputs into one block. Each device applies only the rules whose media queries match it, and since most rules are shared, the combined block is usually only modestly larger than either run on its own.

1

Run the mobile size

Generate at 412 × 823 (or the Mobile preset) and copy the result.

2

Run the desktop size

Switch to 1920 × 1080 and generate again.

3

Combine

Paste the mobile CSS first, then the desktop CSS, into the same `<style>` block or critical CSS field.

NOTE

Duplicate rules in a combined block do no harm: the browser simply applies the last one. If you want to squeeze every byte, remove obvious repeats by hand, but it is rarely worth the effort.

6

ANY WEBSITE

Putting critical CSS to work

Generating critical CSS is half the job. To get the benefit, your page has to inline it and stop the full stylesheets from blocking. Here is the classic pattern, which works on any website, WordPress or not.

```
HTML
<head>
  <!-- 1. The critical CSS, inlined -->
  <style>
    /* paste the generator output here */
  </style>

  <!-- 2. The full stylesheet, loaded without blocking -->
  <link rel="preload" href="/css/style.css" as="style"
        onload="this.onload=null;this.rel='stylesheet'">

  <!-- 3. A fallback for browsers with JavaScript off -->
  <noscript><link rel="stylesheet" href="/css/style.css"></noscript>
</head>
```

How it fits together

- The inlined block styles the first screen the instant the HTML arrives.
- The `preload` link fetches the full stylesheet at high priority without blocking, then switches it on as a normal stylesheet once it has loaded.
- The `noscript` fallback makes sure visitors with JavaScript disabled still get the full styling.

Keeping it fresh

Critical CSS is a snapshot of your design at the moment you generated it. Regenerate it whenever you change the top of the page: a new hero, a redesigned header, a theme update, a new font. If you forget, the worst that usually happens is a brief flash of unstyled or restyled content until the full stylesheet arrives.

WARNING

Never inline critical CSS *and* leave the full stylesheet render-blocking. The browser would still wait for the full file, so you would pay for the extra bytes in your HTML and gain nothing.

On WordPress

Rather than editing theme templates, use a performance plugin that accepts pasted critical CSS and takes care of the non-blocking loading for you. The next section shows how to do this with MBR Performance, which was built with exactly this workflow in mind.

7

THE WORDPRESS ROUTE

Using it with MBR Performance

MBR Performance is a free, open-source WordPress plugin from the same developer. Its CSS tab offers three ways to tackle render-blocking CSS, and output from this generator slots into the third.

Approach	How it works	Best for
Used CSS Mode A	The plugin works out the CSS each page uses, inlines it, and loads the full stylesheets asynchronously behind it. Missed rules are corrected a moment later.	Almost any site. The safe default.
Used CSS Mode B	Learns each template from several sample URLs, inlines what they use, and removes the analysed stylesheets outright.	Sites where every template can be tested on staging first.
Critical CSS (pasted)	You paste CSS from a viewport-aware generator such as this one into per-context slots. When a slot matches the page, your CSS owns delivery for it.	Exact, hand-picked control over a key page or template.

When a pasted Critical CSS slot matches the page being rendered, everything else, including Mode B, stands down on that page, so there is never a clash to manage by hand.

Recommended workflow

1

Ask the Doctor first

Run **Performance Doctor** in MBR Performance. It tells you whether your render-blocking is mostly CSS or mostly JavaScript. If it is JavaScript, the win is on the JavaScript tab, and critical CSS will not move the needle much.

2

Generate

Run this generator for each viewport you care about and combine the results, as described in section 5.

3

Paste

On the CSS tab, paste the CSS into the Critical CSS slot that matches the page or template you generated it for, and save. Saving clears the plugin's page cache, so the next visit is built with your new CSS.

4

Check and measure

View the page logged out in a private window, since some optimisations are skipped for administrators. Then re-test with PageSpeed Insights, and let the plugin's Real User Monitoring confirm that real visitors see the improvement.

TIP

Fonts are the other half of a fast first paint. On the MBR Performance Fonts tab, self-host your Google Fonts, set font-display to swap, and preload only the one or two fonts visible above the fold.

8

MBR-PERFORMANCE.CO.UK

Case study: four 100s

The generator was tested on a real site on its first day: the MBR Performance website, a landing page with six sections, six images and three animated areas, running on shared hosting with MBR Performance installed.

Stage	What happened	PageSpeed
First run	Critical CSS generated and pasted into MBR Performance.	Mobile 99
Desktop run	A true 1920 × 1080 run produced a proper desktop set.	Desktop 100
Combined	Mobile (412 × 823) and desktop outputs joined into one block.	Desktop 100, mobile 99
Detective work	The LCP breakdown showed 0 ms time to first byte but 1,280 ms of element render delay on the hero headline.	Cause found
The fix	The headline's entrance animation was switched off with three lines of CSS.	Mobile 100

The final result: 100 for Performance, Accessibility, Best Practices and SEO on mobile, and the same on desktop.

The lesson: watch your entrance animations

The last point had nothing to do with CSS delivery. The hero headline faded in from `opacity: 0`, and Chrome does not count an element as painted while it is invisible. So the largest element on screen, the headline, was only recorded once the animation had played, adding over a second to LCP. The fix kept every other animation on the page and simply let the headline appear at once:

```
css
.hero__title {
  animation: none;
  opacity: 1;
  transform: none;
}
```

CASE STUDY

If PageSpeed shows a large **element render delay** for your LCP element while time to first byte is tiny, look for an animation, transition or script that hides that element at load. It is the most common reason a well-optimised page stops just short of 100.

9

GETTING THE BEST RESULTS

Tips, limits and good habits

Things the generator cannot see

Because scripts never run during rendering, anything JavaScript creates or reveals after the page loads is invisible to the generator. In practice that means:

- **Script-built layouts.** Sliders that build their slides with JavaScript, and fully client-rendered apps, may give limited results. The page you see in the preview is the page the generator measured.
- **Late arrivals.** Cookie banners, chat bubbles, sticky headers that switch class on scroll, and modals are not in the critical set. That is usually fine: they appear after the first paint, by which time the full stylesheet has loaded.
- **Embedded frames.** Videos and other iframes keep their size in the layout but are not loaded, so their contents are not measured.

Web fonts

Some sites serve their font files without the cross-origin headers a browser needs to load them inside another site's page. When that happens, the generator measures the layout with fallback fonts, which can shift what lands above the fold by a line or two. The `@font-face` rules are still included in your output, so the real fonts load normally on your own site.

Good habits

- Generate from the live, public version of the page, with any page cache warmed.
- Check the preview before copying. If it does not look like your page, the CSS will not either.
- Read any notes. They explain skipped stylesheets and size limits in plain English.
- Regenerate after design changes to the top of the page, theme updates, or new fonts.
- Test logged out, in a private window, after every change.

Fair-use limits

To keep the tool free and fast for everyone, each visitor can run a limited number of generations in a short window, and only a few generations run at once across all visitors. If it is busy, wait a few seconds and try again; if you hit your own limit, the message says how many minutes to wait.

Limit	Per generation
Time	About 30 seconds of fetching and processing
Page HTML	3 MB
Stylesheet requests	60, including @import files and any that fail
CSS downloaded	2 MB per file, 10 MB in total
CSS processed	12 MB, counting repeated stylesheets and URLs that grow when made absolute
@import rules	100, with circular imports detected and skipped

These are far above what a normal web page uses.

10

ANSWERS

Troubleshooting & FAQ

It says the address cannot be fetched.

The generator can only reach public websites on standard ports. Addresses on a private network, localhost, or a staging site behind a password are refused deliberately. Some sites also block automated requests through a firewall or bot protection; if so, try a different page, or ask the site owner to allow the request.

The result is very small, or empty.

The page probably builds its layout with JavaScript, which does not run during rendering. Check the preview: if it is mostly blank, the generator had little to measure.

A note says the page rendered at a different size.

Something on the page is resizing the preview frame, so the result may not match the viewport you chose. Treat that run with caution, and if it keeps happening, get in touch with the URL.

A note says a stylesheet was skipped.

It could not be fetched, returned a web page instead of CSS, or pointed somewhere private. Rules from that file will not be in the output. If it matters, check the file loads in your own browser.

My page still flashes when it loads.

Check that the full stylesheet is no longer render-blocking, that you generated at the right viewport, and that you regenerated after recent design changes. Also make sure no other cache is serving an old copy of the page.

It says the generator is busy.

Only a few generations run at the same time, to keep the service responsive. Wait a few seconds and press Generate again. Each attempt counts towards your own allowance, so there is no need to retry rapidly.

The layout & style check names elements that differ.

A small number of minor differences is normal, often from fonts that load at slightly different moments. If a large element such as the header or hero is listed, compare the preview with your live page, and check the notes for stylesheets that were skipped.

A note says references were removed.

An image, font or media address on the page pointed at a private network, a local address or this generator's own site, so it was blocked for safety. In the output it is replaced with an empty placeholder; if it should be there, fix the address on your page and generate again.

Does the generator keep a copy of my page or CSS?

No. The page is fetched, handed to your browser, and discarded. The CSS is produced in your browser and never sent back. The only thing stored is a short-lived usage counter for the fair-use limit, keyed to a scrambled (hashed) form of your IP address and deleted within a day.

Is it really free?

Yes. Like everything from Little Web Shack, there is no paid tier, no signup and no tracking.

11

JARGON, DECODED

Glossary

Term	Meaning
Above the fold	The part of a page visible without scrolling, on a given screen size.
Async CSS	Loading a stylesheet so it does not block the first paint, typically with a preload link or a print-media swap.
Core Web Vitals	Google's three user-experience metrics: LCP, INP and CLS.
Critical CSS	The CSS rules needed to draw what is visible above the fold, inlined in the page head.
CLS	Cumulative Layout Shift: how much the page jumps around while it loads. Good is under 0.1.
FCP	First Contentful Paint: when the first text or image appears.
Layout & style check	The generator's own test: it renders the page with only the critical CSS and compares layout and key styles in the first view against the full render.
Field data	Measurements from real visitors, as opposed to a test run. See RUM.
The fold	The bottom edge of the viewport when the page first loads.
INP	Interaction to Next Paint: how quickly the page responds to taps, clicks and typing.
LCP	Largest Contentful Paint: when the biggest visible element finishes drawing. Good is under 2.5 seconds.
Media query	A CSS condition such as <code>@media (max-width: 768px)</code> that applies rules only on certain screens.
Minification	Removing whitespace and redundant characters from CSS without changing what it does.
Render-blocking	A resource the browser must finish loading before it can paint anything.
RUM	Real User Monitoring: collecting Core Web Vitals from actual visitors.
Sandboxed frame	A frame in which scripts cannot run. The generator renders pages this way so nothing on the fetched page can execute.
Viewport	The visible area of the browser window, measured in CSS pixels.

12

FOR SITE OWNERS AND AGENCIES

Host your own generator

The generator at mbr-critical.co.uk is a free WordPress plugin, and you can run your own copy: on your agency site, a client portal, or an internal tools page. It works on ordinary shared hosting, with nothing to install beyond the plugin.

You need	Details
WordPress 6.0+	Tested up to WordPress 7.0.
PHP 7.4+	With the DOM extension, which almost every host includes as standard.
Outgoing HTTPS	Your server must be allowed to fetch other websites. Most hosts permit this.

1

Install

Download **MBR Critical CSS Generator** from littlewebshack.com, then go to **Plugins → Add New → Upload Plugin**, choose the ZIP and activate it. Future versions arrive on the Plugins screen like any other update.

2

Add the shortcode

Add the shortcode below to any page. The optional **width** and **height** set the starting viewport. A wide or full-width template gives the preview the most room.

3

Configure

Visit **Settings → Critical CSS Generator** to set who can use it and how hard it can be used (see below).

SHORTCODE

```
[mbr_critical_css]
```

```
[mbr_critical_css width="375" height="812"]
```

Setting	Default	What it does
Access	Everyone	Tick to make the generator available to logged-in users only.
Rate limit	10 per 10 min	Generations allowed per visitor IP address. Set 0 to switch it off. Administrators are never limited.
Simultaneous jobs	3	How many generations run at once across all visitors. Each can occupy a PHP worker for up to about 30 seconds, so keep it well below your hosting plan's limit.

TIP

Page caching is fine on the generator page. For a busy public tool, rate limiting at your host or CDN adds an extra layer. To match your brand, set the accent colour in Additional CSS:

```
.mbr-ccss { --mbr-ccss-accent: #7a3e9d; }
```

13

MAKE WORDPRESS FAST

Don't have MBR Performance yet?

Critical CSS is one technique. MBR Performance brings together everything else a WordPress site needs to load fast, and shows you where to start. It is free and open source, with no premium tier.

Feature	What it does for you
Performance Doctor	Scans a real page, or your key templates at once, and lists in priority order which settings will help, which to leave alone, and where to find them.
Advanced Page Cache	Saves finished pages and serves them to later visitors. Where your server allows, pages are served as static files without starting PHP at all.
50+ optimisations	Granular control over scripts, styles, fonts, images, lazy loading, the database, server headers and WooCommerce. Enable only what suits your site.
Used CSS and Critical CSS	Two automatic modes for removing render-blocking CSS, plus slots for critical CSS from this generator.
Real User Monitoring	Collects LCP, INP and CLS from real visitors and stores them in your own database. No cookies, no IP addresses, nothing leaves your server.
A 45-page User Guide	Every setting explained in plain English, so each switch makes sense.

Download MBR Performance, free

Install it, run the Doctor, and let real-visitor data prove the difference. Read the full User Guide on the website before you install, if you like.

mbr-performance.co.uk

14

FREE MEANS FREE

More from Little Web Shack

Little Web Shack is a small workshop of security-first WordPress plugins and browser-based developer tools, built and maintained by Robert Palmer in Cleethorpes, UK. A growing collection of plugins and dev tools, and zero paywalls.

WordPress plugins

Plugin	What it does
MBR Performance	All-in-one speed: page cache, 50+ optimisations, Performance Doctor and Real User Monitoring.
MBR Critical CSS Generator	This generator, as a plugin: host your own copy with one shortcode.
MBR Cookie Consent	Consent management with Google Consent Mode v2 and 20+ languages.
MBR Intelligent Site Assistant	On-site conversational search built for shared hosting, with no external AI API.
MBR Advanced Asset Manager	Choose exactly which stylesheets and scripts load on each page, post or URL pattern. A natural partner to critical CSS.
MBR Bulk WP Detector	Scan lists of URLs to detect WordPress, themes, plugins, page builders and widgets.
MBR Live Radio Player	Turn a WordPress site into a radio station with a sticky player.

Also in the collection: **Login Customiser**, **HTML Generator**, **Elementor Form Icons** and **WP Site Detector**.

Browser-based dev tools

Tool	What it does
Glassmorphism Generator	Dial in blur, opacity, saturation, tint and border, then copy production CSS.
Glass Text Generator	Refraction, depth and an optional animated shimmer on live type.
Font Clamp Generator	A mathematically exact <code>clamp()</code> for fluid type, with a live viewport simulator.
Theme Generator	Build and export a complete WordPress theme as a ready-to-install ZIP.

Explore the whole collection

No upsells, no premium versions, no telemetry. Every plugin is GPL-licensed, with the source on GitHub at github.com/HarbourBob. Questions, ideas, or a page that gives odd results? Get in touch through the contact page.

littlewebshack.com